

APDL_Input.py

```
1  ### ADPL_Input.py ###
2  # -> INPUT:
3  #     - [SW_coor]   -> Coordinates from Soldiworks
4  #     - [var]       -> Radius variables
5  #     - [Misc]      -> Miscellaneous Data (force, mesh etc.)
6  # -> OUTPUT:
7  #     - .txt Input file for APDL Analysis (Eigenbuckling and Nonlinear)
8
9  # Pseudo Code
10     # Input is coordinate list with format [(x1,y1,z1),(x2,y2,z3)]
11     # Create Two Keypoints with these coordinates and create a line with them
12     #     K, ID, X, Y, Z
13     #     L, P1, P2
14     # Sort lines into two groups:
15     #     1. Vertical / Corner beams, that only varies in y direction
16     #     2. Brace beams, so everyone else
17     # Define element type with ET,1,BEAM189
18     # Define Cross Section
19     # Define material properties
20     # Apply SECTYPE,1 to vertical beams
21     # Apply SECTYPE,2 to brace beams
22     # Apply SECTYPE,3 to Top (constant)
23
24     # Import packages
25     import os
26     import SW_Import as SW
27
28
29     def InputFun(var,Misc,opti_settings):
30         SW_filename = Misc["SW_filename"]
31
32         # Initialize APDL Command for PyMAPDL
33         SWcoor = SW.import_SW(os.path.join("IGS",SW_filename))
34         ap = []
35         # Only use Rad if selected in main
36         rad = var["rad"]["value"] if "rad" in var and var["rad"].get("active", True) else None
37
38         n = opti_settings["n_mast_segments"]
39         mast_height = opti_settings["mast_segment_height"]
40         brace_start = n + 1
41         top_sectype = 99
42
43         # Convert to Radii
44         if opti_settings.get("multi_size_columns", True):
45             R0_list = [var[f"d0_{i}"]["value"]/2 - var[f"t0_{i}"]["value"] for i in range(1, n+1)]
46             R1_list = [var[f"d0_{i}"]["value"]/2 for i in range(1, n+1)]
47         else:
48             R0 = var["d0"]["value"]/2 - var["t0"]["value"]
49             R1 = var["d0"]["value"]/2
50
51         if opti_settings.get("multi_size_braces", True):
52             if opti_settings.get("brace_split", False):
53                 # Horizontal braces
54                 R2_horiz_list = [var[f"d1_h_{i}"]["value"]/2 - var[f"t1_h_{i}"]["value"] for i in range(1, n+1)]
55                 R3_horiz_list = [var[f"d1_h_{i}"]["value"]/2 for i in range(1, n+1)]
56                 # Cross braces
57                 R2_cross_list = [var[f"d1_c_{i}"]["value"]/2 - var[f"t1_c_{i}"]["value"] for i in range(1, n+1)]
58                 R3_cross_list = [var[f"d1_c_{i}"]["value"]/2 for i in range(1, n+1)]
59             else:
60                 # Combined braces (same size for both horizontal and cross)
61                 R2_list = [var[f"d1_{i}"]["value"]/2 - var[f"t1_{i}"]["value"] for i in range(1, n+1)]
62                 R3_list = [var[f"d1_{i}"]["value"]/2 for i in range(1, n+1)]
```

```

63     else:
64         if opti_settings.get("brace_split", False):
65             R2_horiz = var["d1_h"]["value"]/2 - var["t1_h"]["value"]
66             R3_horiz = var["d1_h"]["value"]/2
67             R2_cross = var["d1_c"]["value"]/2 - var["t1_c"]["value"]
68             R3_cross = var["d1_c"]["value"]/2
69         else:
70             R2 = var["d1"]["value"]/2 - var["t1"]["value"]
71             R3 = var["d1"]["value"]/2
72
73     # Change Radius Logic
74     if rad is not None:
75
76         import math
77
78         # Check if Column Member
79         def is_column(x1, z1, x2, z2):
80             return (x1 == x2) and (z1 == z2)
81
82         # Only take Column Members
83         col_loc = {(x1, z1) for x1, y1, z1, x2, y2, z2 in SWcoor if is_column(x1, z1, x2, z2)}
84
85         loc_to_new = {}
86         for (x, z) in col_loc:
87             # r0 = \sqrt(x^2+z^2)
88             r0 = math.hypot(x, z)
89             # Scale radius
90             s = rad / 202.07 # NEEDS TO CHANGE IF ANOTHER IGS FILE IS USED
91             # Apply scaling to x and z values
92             loc_to_new[(x, z)] = (x * s, z * s)
93
94         def adjust(x, y, z):
95             if (x,z) in loc_to_new:
96                 nx,nz = loc_to_new[(x,z)]
97                 return nx,y,nz
98             return x, y, z
99
100         SWcoor = [
101             (*adjust(x1, y1, z1), *adjust(x2, y2, z2))
102             for x1, y1, z1, x2, y2, z2 in SWcoor
103         ]
104
105
106     # Function to group lines
107     def beam_class(p1, p2):
108
109         (x1, y1, z1) = p1
110         (x2, y2, z2) = p2
111
112         tol = 1e-3 # Tolerance for floating point comparison
113
114         # Corner beam if x2 == x1 AND z2 == z1, i.e. these are unchanged.
115         if abs(x1 - x2) < tol and abs(z1 - z2) < tol:
116             return "corner"
117         # Horizontal brace if y coordinates are the same
118         elif abs(y1 - y2) < tol:
119             return "horizontal"
120         # Cross brace if y coordinates differ
121         else:
122             return "cross"
123
124

```

```

125         # SETUP
126         ap.append("! ===== APDL INPUT FILE ===== ! ")
127         #####
128         ##### Eigenbuckling Analysis #####
129         #####
130         ap.append("! Eigenbuckling Analysis ! ")
131         ap.append("! ===== ! ")
132         ap.append("/UNITS,MPa ! Set units [mm,Mg,s,C] ")
133
134         ap.append("! Static Structural Analysis ! ")
135         ap.append("/PREP7 ")
136         ap.append("ET,1,BEAM189 ! Use BEAM189 Elements ")
137         ## CROSS SECTION
138         ap.append("! CROSS SECTION ! ")
139         if opti_settings.get("multi_size_columns", True):
140             for i in range(1, n+1):
141                 ap.append(f"SECTYPE,{i},BEAM,CTUBE ")
142                 ap.append(f"SECDATA,{R0_list[i-1]},{R1_list[i-1]},8 ")
143         else:
144             ap.append("SECTYPE,1,BEAM,CTUBE ")
145             ap.append(f"SECDATA,{R0},{R1},8 ")
146
147         if opti_settings.get("multi_size_braces", True):
148             if opti_settings.get("brace_split", False):
149                 # Horizontal braces
150                 for i in range(1, n+1):
151                     secnum = brace_start + 2*(i-1)
152                     ap.append(f"SECTYPE,{secnum},BEAM,CTUBE ")
153                     ap.append(f"SECDATA,{R2_horiz_list[i-1]},{R3_horiz_list[i-1]},8 ")
154                 # Cross braces
155                 for i in range(1, n+1):
156                     secnum = brace_start + 2*(i-1) + 1
157                     ap.append(f"SECTYPE,{secnum},BEAM,CTUBE ")
158                     ap.append(f"SECDATA,{R2_cross_list[i-1]},{R3_cross_list[i-1]},8 ")
159             else:
160                 # Combined braces (same size for both horizontal and cross)
161                 for i in range(1, n+1):
162                     secnum = brace_start + i - 1
163                     ap.append(f"SECTYPE,{secnum},BEAM,CTUBE ")
164                     ap.append(f"SECDATA,{R2_list[i-1]},{R3_list[i-1]},8 ")
165         else:
166             if opti_settings.get("brace_split", False):
167                 # Horizontal braces
168                 ap.append(f"SECTYPE,{brace_start},BEAM,CTUBE ")
169                 ap.append(f"SECDATA,{R2_horiz},{R3_horiz},8 ")
170                 # Cross braces
171                 ap.append(f"SECTYPE,{brace_start+1},BEAM,CTUBE ")
172                 ap.append(f"SECDATA,{R2_cross},{R3_cross},8 ")
173             else:
174                 ap.append(f"SECTYPE,{brace_start},BEAM,CTUBE ")
175                 ap.append(f"SECDATA,{R2},{R3},8 ")
176
177         ap.append(f"SECTYPE,{top_sectype},BEAM,CTUBE ")
178         ap.append("SECDATA,35.05,38.05,8 ")
179
180         # MATERIAL
181         ap.append("! MATERIAL DATA ")
182         ap.append(f"MP,EX,1,{Misc['E_mod']} ! [MPa]")
183         ap.append("MP,PRXY,1,0.3 ")
184         ap.append(f"MP,DENS,1,7.850E-6 ! [kg/mm^3]")
185
186         # STIFF Material

```

APDL_Input.py

```

187 ap.append("! INF STIFNESS MATERIAL REGION ABOVE Y=4078 ")
188 ap.append("MP,EX,2,2E+09 ")
189 ap.append("MP,PRXY,2,0.3 ")
190 ap.append("MP,DENS,2,7.850E-6 ")
191
192 # NODES
193 ap.append("! KEYPOINT AND LINES ! ")
194 # Initialize values used later
195 key_id = 1
196 line_id = 1
197 if opti_settings.get("multi_size_columns", True):
198     corner_lines = [[] for _ in range(n)]
199 else:
200     corner_lines = []
201 if opti_settings.get("multi_size_braces", True):
202     horiz_lines = [[] for _ in range(n)]
203     cross_lines = [[] for _ in range(n)]
204 else:
205     horiz_lines = []
206     cross_lines = []
207 segment_lines = [[] for _ in range(n)]
208 Top_lines = []
209 corner_id = 1
210 horiz_id = 1
211 cross_id = 1
212 Top_lines = []
213
214 kp_dict = {}
215 CM_Brace_dict = 0
216 CM_Column_dict = 0
217
218
219 # Loop through and create lines
220 for x1, y1, z1, x2, y2, z2 in SWcoor:
221
222     # First point
223     p1 = (x1, y1, z1)
224     if p1 in kp_dict: # (We check if the point already exists)
225         kp1 = kp_dict[p1]
226     else:
227         kp1 = key_id
228         kp_dict[p1] = kp1
229         ap.append(f"K,{kp1}, {x1:.6f}, {y1:.6f}, {z1:.6f} ")
230         key_id += 1
231
232     # Second Point
233     p2 = (x2, y2, z2)
234     if p2 in kp_dict: # (We check if the point already exists)
235         kp2 = kp_dict[p2]
236     else:
237         kp2 = key_id
238         kp_dict[p2] = kp2
239         ap.append(f"K,{kp2}, {x2:.6f}, {y2:.6f}, {z2:.6f} ")
240         key_id += 1
241
242     # Group the lines points
243     group = beam_class(p1,p2)
244     # Create line
245     ap.append(f"L,{kp1},{kp2} ")
246     ap.append(f"LSEL,S,LINE,,{line_id} ")
247
248

```

```

249     # Split lines into corner, brace or top section
250     if y1 > 4078 and y2 > 4078:
251         # This is a TOP beam!
252         Top_lines.append(line_id)
253         ap.append(f"CM,TOPMAT,LINE ")
254
255     else:
256         # Not a top beam ? classify as corner, horizontal or cross brace
257         if group == "corner":
258             segment = int(min(y1, y2) // mast_height) + 1
259             segment = min(max(segment, 1), n)
260             if opti_settings.get("multi_size_columns", True):
261                 corner_lines[segment-1].append(line_id)
262             else:
263                 corner_lines.append(line_id)
264                 segment_lines[segment-1].append(line_id)
265                 ap.append(f"CM,COLUMN_{corner_id},LINE ")
266                 corner_id += 1
267                 CM_Column_dict += 1
268         elif group == "horizontal":
269             segment = int(min(y1, y2) // mast_height) + 1
270             segment = min(max(segment, 1), n)
271             if opti_settings.get("multi_size_braces", True):
272                 horiz_lines[segment-1].append(line_id)
273             else:
274                 horiz_lines.append(line_id)
275                 segment_lines[segment-1].append(line_id)
276                 ap.append(f"CM,HORIZ_{horiz_id},LINE ")
277                 horiz_id += 1
278                 CM_Brace_dict += 1
279         else: # cross
280             segment = int(min(y1, y2) // mast_height) + 1
281             segment = min(max(segment, 1), n)
282             if opti_settings.get("multi_size_braces", True):
283                 cross_lines[segment-1].append(line_id)
284             else:
285                 cross_lines.append(line_id)
286                 segment_lines[segment-1].append(line_id)
287                 ap.append(f"CM,CROSS_{cross_id},LINE ")
288                 cross_id += 1
289                 CM_Brace_dict += 1
290     # Reset
291     ap.append("LSEL,ALL ")
292
293     line_id += 1
294     ap.append(" ")
295
296     # ELEMENT DEFINITION
297     ap.append("! ELEMENT SIZE ! ")
298     ap.append(f"ESIZE,,{Misc['esize']} ")
299
300     # Function to mesh each group seperatly
301     # Makes sure each section has the correct cross section
302     def group_mesh(block_name, secnum, line_ids):
303
304         ap.append(f"! {block_name} ! ")
305         ap.append(f"SECNUM,{secnum} ")
306         ap.append("LSEL,ALL ")
307
308         first = True
309         for lid in line_ids:
310             if first:

```

APDL_Input.py

```

311         ap.append(f"LSEL,S,LINE,,{lid} ")
312         first = False
313     else:
314         ap.append(f"LSEL,A,LINE,,{lid} ")
315
316     ap.append("LMESH,ALL ")
317
318
319     # Save brace counts
320     CM_dict = [CM_Column_dict, horiz_id - 1, cross_id - 1] # Brace counts (horizontal, cross)
321
322
323     # Run the function for Corner and Brace
324     if opti_settings.get("multi_size_columns", True):
325         for i in range(n):
326             if corner_lines[i]:
327                 group_mesh(f"Meshing CORNER Beams Segment {i+1} (SECNUM={i+1})", i+1, corner_lines[i])
328     else:
329         group_mesh("Meshing CORNER Beams (SECNUM=1)", 1, corner_lines)
330
331     if opti_settings.get("multi_size_braces", True):
332         if opti_settings.get("brace_split", False):
333             for i in range(n):
334                 # Horizontal braces
335                 if horiz_lines[i]:
336                     secnum = brace_start + 2*i
337                     group_mesh(f"Meshing HORIZ Beams Segment {i+1} (SECNUM={secnum})", secnum, horiz_lines[i])
338                 # Cross braces
339                 if cross_lines[i]:
340                     secnum = brace_start + 2*i + 1
341                     group_mesh(f"Meshing CROSS Beams Segment {i+1} (SECNUM={secnum})", secnum, cross_lines[i])
342         else:
343             # Combined braces - same section number for both horizontal and cross
344             for i in range(n):
345                 if horiz_lines[i]:
346                     secnum = brace_start + i
347                     group_mesh(f"Meshing HORIZ Beams Segment {i+1} (SECNUM={secnum})", secnum, horiz_lines[i])
348                 if cross_lines[i]:
349                     secnum = brace_start + i
350                     group_mesh(f"Meshing CROSS Beams Segment {i+1} (SECNUM={secnum})", secnum, cross_lines[i])
351     else:
352         if opti_settings.get("brace_split", False):
353             group_mesh(f"Meshing HORIZ Beams (SECNUM={brace_start})", brace_start, horiz_lines)
354             group_mesh(f"Meshing CROSS Beams (SECNUM={brace_start+1})", brace_start+1, cross_lines)
355         else:
356             # Combine all braces (horizontal + cross) into one section
357             combined_lines = horiz_lines + cross_lines
358             group_mesh(f"Meshing ALL Braces (SECNUM={brace_start})", brace_start, combined_lines)
359
360     group_mesh(f"Meshing TOP Beam (SECNUM={top_sectype})", top_sectype, Top_lines)
361
362     # Select all lines
363     ap.append("LSEL,ALL ")
364
365     # Select all 'top material' lines again
366     first = True
367     for lid in Top_lines:
368         if first:
369             ap.append(f"LSEL,S,LINE,,{lid} ")
370             first = False
371         else:
372             ap.append(f"LSEL,A,LINE,,{lid} ")

```

```

373
374 ap.append("ESLL,S ! select elements on these lines ")
375 ap.append("EMODIF,ALL,MAT,2 ! modify selected elements to material 2 ")
376 ap.append("ALLSEL,ALL ")
377
378 # Select all nodes with x=0
379 ap.append("SELTOL,1.0E-6 ") # Important for node selection
380 ap.append("NSEL,S,LOC,X,0")
381 ap.append("NSEL,R,LOC,Z,0")
382 ap.append("NSEL,U,LOC,Y,4283,6000") # Old value: 4282
383 ap.append("*GET,SlaveNum,NODE,0,COUNT")
384
385 ap.append("*DIM,SlaveIDs,ARRAY,SlaveNum")
386 ap.append("*VGET,SlaveIDs(1),NODE,,NLIST") # Stores all node IDs
387
388 # Create Master / Independent Node
389 #ap.append("N,99999,0,4.179140091E+03,0")
390 ap.append("N,99999,0,4180.14,0 ") # Old value: 4182.1384
391 ap.append("*SET,tid,11")
392 ap.append("*SET,cid,10")
393 ap.append("ET,cid,175")
394 ap.append("ET,tid,170")
395 ap.append("KEY0,tid,2,1")
396 ap.append("KEY0,tid,4,0")
397 ap.append("KEY0,cid,12,5")
398 ap.append("KEY0,cid,4,0")
399 ap.append("KEY0,cid,2,2")
400 ap.append("MAT,10")
401 ap.append("REAL,10")
402 ap.append("TYPE,10")
403
404 ap.append("*DO,ii,1,SlaveNum,1")
405 ap.append("    *SET,elemID,8999+ii")
406 ap.append("    *SET,nodeID,SlaveIDs(ii)")
407 ap.append("    EN,elemID,nodeID")
408 ap.append("*ENDDO")
409
410 # Pilot Node Options
411 ap.append("*SET,_npilot,99999")
412 ap.append("_npilot1=_npilot")
413 ap.append("TYPE,tid")
414 ap.append("MAT,cid")
415 ap.append("REAL,cid")
416 ap.append("TSHAPE,PILO")
417 ap.append("EN,79999,_npilot")
418 ap.append("TSHAPE")
419 # Display Cross section
420 ap.append("ALLSEL")
421
422 #Create and save .png of the mesh
423 #ap.append("/SHOW,PNG,,0 ")
424 #ap.append("/RGB,INDEX,100,100,100,0 ")
425 #ap.append("/RGB,INDEX,80,80,80,13 ")
426 #ap.append("/RGB,INDEX,60,60,60,14 ")
427 #ap.append("/RGB,INDEX,0,0,0,15 ")
428 #ap.append("/TYPE,,4 ")
429 #ap.append("/VIEW,,0,0,1 ")
430 #ap.append("/ANGLE,,30,YM ")
431 #ap.append("EPLLOT ")
432 #ap.append("/SHOW,close ")
433 #ap.append("/SHOW,TERM ")
434

```

```

435
436 # RUN STATIC ANALYSIS
437 # We use sparse solver with pre-stress on
438 ap.append("! SOLUTION ! ")
439 ap.append("/SOLU ")
440 ap.append("ANTYPE, STATIC ")
441 ap.append("EQSLV, SPARSE ")
442 ap.append("PSTRES, ON ")
443
444 # BOUNDARY CONDITIONS
445 ap.append(" ! -- BOUNDARY CONDITIONS -- ! ")
446 ap.append("ALLSEL, ALL ")
447 ap.append("SELTOL, 1.0E-6 ") # Important for node selection
448
449 # Get top and bottom nodes
450 ap.append("*GET, NodeYMax, NODE, 0, MXLOC, Y ")
451 ap.append("*GET, NodeYMin, NODE, 0, MNLOC, Y ")
452 ap.append("*GET, NodeXMax, NODE, 0, MXLOC, X ")
453
454 ap.append("NSEL, S, LOC, Y, NodeYMax")
455 ap.append("NSEL, R, LOC, X, NodeXMax")
456 ap.append(f"F, ALL, FY, {Misc['Ver_Force']}")
457 ap.append(f"F, ALL, FZ, {Misc['Hor_Force']}")
458 ap.append("NSEL, ALL")
459
460 # Force at x = 0 / Weight
461 ap.append("NSEL, S, NODE, , 99999")
462 ap.append(f"F, ALL, FY, {Misc['W_Force']}")
463
464 # Fixed displacement at bottom nodes
465 ap.append("! Displacement ! ")
466 ap.append("ALLSEL, ALL ")
467 ap.append("NSEL, S, LOC, Y, NodeYMin ")
468 ap.append("D, ALL, ALL, 0 ")
469 ap.append("ALLSEL ")
470
471 # SOLVE
472 ap.append("! Solve the System ")
473 ap.append("SOLVE ")
474 ap.append("FINISH ")
475
476 # MASS OF ASSEMBLY
477 ap.append("! Get and Print Mass ")
478 ap.append("ALLSEL ")
479 ap.append("NSEL, S, LOC, Y, , 4050 ")
480 ap.append("ESLN ")
481 ap.append("*GET, ecnt, ELEM, 0, COUNT ")
482 ap.append("*GET, enum, ELEM, 0, NUM, MIN ")
483 ap.append("totvol = 0 ")
484
485 # Loop over each element and get volume
486 ap.append("*DO, i, 1, ecnt ")
487 ap.append(" *GET, ev, ELEM, enum, VOLU ")
488 ap.append(" totvol = totvol + ev ")
489 ap.append(" enum = ELNEXT(enum) ")
490 ap.append("*ENDDO ")
491
492 # Calculate Mass
493 ap.append("dens = 7.85E-6 ! kg/mm^3 ") # Density
494 ap.append("Comp_mass = dens*totvol ")
495
496 # Open and write to file

```



```

497 ap.append("*CFOPEN,MASS_assembly,txt ")
498 ap.append(" *VWRITE,Comp_mass ")
499 ap.append(" (F12.5) ")
500 ap.append("*CFCLOSE ")
501 ap.append("FINISH ")
502 ap.append("ALLSEL,ALL ")
503
504 # Mass of Each Segment
505 for i in range(1, n+1):
506     ap.append(f"! Get Mass of Segment {i} ")
507     ap.append("ALLSEL ")
508     first = True
509     for lid in segment_lines[i-1]:
510         if first:
511             ap.append(f"LSEL,S,LINE,,{lid} ")
512             first = False
513         else:
514             ap.append(f"LSEL,A,LINE,,{lid} ")
515     ap.append("ESLL,S ")
516     ap.append("*GET,ecnt,ELEM,0,COUNT ")
517     ap.append("*GET,enum,ELEM,0,NUM,MIN ")
518     ap.append("seg_vol = 0 ")
519     ap.append("*DO,j,1,ecnt,1 ")
520     ap.append(" *GET,ev,ELEM,enum,VOLU ")
521     ap.append(" seg_vol = seg_vol + ev ")
522     ap.append(" enum = ELNEXT(enum) ")
523     ap.append("*ENDDO ")
524     ap.append(f"seg_mass_{i} = dens*seg_vol ")
525
526 ap.append("*CFOPEN,MASS_segments,txt ")
527 for i in range(1, n+1):
528     ap.append(f" *VWRITE,seg_mass_{i} ")
529     ap.append(" (F20.6) ")
530 ap.append("*CFCLOSE ")
531 ap.append("FINISH ")
532 ap.append("ALLSEL,ALL ")
533
534 # EIGENBUCKLING
535 ap.append("! Eigenbuckling Solution! ")
536 ap.append("/SOLU ")
537 ap.append("ANTYPE,BUCKLE ")
538 ap.append("BUCOPT,LANB,10 ")
539 ap.append("MPXAND,ALL ")
540 ap.append("OUTRES,ALL,ALL ")
541 ap.append("SOLVE ")
542 ap.append("FINISH ")
543
544 ap.append("! Retrieve first 10 eigenvalues ")
545 ap.append("/POST1 ")
546 ap.append("*CFOPEN,Eigenvalue1,txt ")
547 ap.append("*DO,jj,1,10,1 ")
548 ap.append(" *GET,MS%jj%,MODE,jj,FREQ ")
549 ap.append(" *VWRITE,MS%jj% ")
550 ap.append(" (F15.5) ")
551 ap.append("*ENDDO ")
552 ap.append("*CFCLOSE ")
553
554 # Calculate Imperfection Force
555 ap.append("ALLSEL ")
556 ap.append("*GET,N_HIGHEST_ret,NODE,0,MXLOC,Y ")
557 ap.append("N_HIGHEST = N_HIGHEST_ret*1E-3 ")
558 ap.append("alpha_h = 2/sqrt(N_HIGHEST) ")

```

APDL_Input.py

```

559     ap.append("IF,alpha_h,LT,0.66,THEN ")
560     ap.append("    alpha_h = 0.66 ")
561     ap.append("ELSEIF,alpha_h,GT,1,THEN ")
562     ap.append("    alpha_h = 1 ")
563     ap.append("ENDIF ")
564     ap.append("alpha_m = 2 ! Assumed for now ")
565     ap.append("imp_ang = 1/200 * alpha_h * alpha_m ")
566     ap.append(f"FORCE_IMP = {Misc['Ver_Force']}*imp_ang ")
567
568     #ap.append("ALLSEL,ALL    FDELE,ALL,ALL    DDELE,ALL,ALL ")
569
570     #####
571     ##### Nonlinear Analysis #####
572     #####
573
574     # NONLINEAR ANALYSIS SETTINGS
575     ap.append("! SOLUTION ! ")
576     ap.append("/SOLU ")
577     ap.append("ANTYPE, STATIC ")
578     ap.append("NROPT,FULL")
579     ap.append("EQLV,SPARSE")
580     ap.append("NLGEOM,ON ")
581     #ap.append("cnvtol,f,,0.005,,0.01")
582     #ap.append("ARCLN,ON")
583     #ap.append("ARCTRM,L")
584     #ap.append("AUTOTS,OFF ")
585     #ap.append("NSUBST,100,100,100")
586     #ap.append("time,1.")
587     #ap.append("NEQIT,200")
588
589     # Get top and bottom nodes
590     ap.append("GET, NodeYMax, NODE, 0, MXLOC, Y ")
591     ap.append("GET, NodeYMin, NODE, 0, MNLOC, Y ")
592     ap.append("GET, NodeXMax, NODE, 0, MXLOC, X ")
593
594
595     ap.append("NSEL,S,LOC,Y,NodeYMax")
596     ap.append("NSEL,R,LOC,X,NodeXMax")
597     ap.append(f"F,ALL,FY,{Misc['Ver_Force']}")
598     ap.append(f"F,ALL,FZ,{Misc['Hor_Force']}")
599     ap.append("NSEL,ALL")
600
601     # Force at x = 0 / Weight
602     ap.append("NSEL,S,NODE,,99999")
603     ap.append(f"F,ALL,FY,{Misc['W_Force']}")
604     ap.append(f"F,ALL,FX,FORCE_IMP")
605
606     # Fixed Displacement at Bottom Nodes
607     ap.append("! Displacement ! ")
608     ap.append("ALLSEL,ALL ")
609     ap.append("NSEL,S,LOC,Y,NodeYMin ")
610     ap.append("D,ALL,ALL,0 ")
611     ap.append("ALLSEL ")
612
613     # SOLVE
614     ap.append("! Solve the System ")
615     ap.append("SOLVE ")
616     ap.append("FINISH ")
617
618     ap.append("! ===== APDL OUTPUT FILE ===== ! ")
619
620     ap.append("/POST1 ")

```

```

621 ap.append("SET, LAST ")
622 ap.append("ALLSEL, ALL ")
623
624 # ONLY SELECT BEAM189 ELEMENTS
625 ap.append("*GET, E_COUNT, ELEM, 0, COUNT ")
626 ap.append(f"! Number of Columns: {CM_dict[0]} ")
627 ap.append(f"! Number of Horiz Braces : {CM_dict[1]} ")
628 ap.append(f"! Number of Cross Braces : {CM_dict[2]} ")
629 # SET OUTPUT FILE
630 ap.append("! Open file to write ")
631 ap.append("CFOPEN, APDL_Nonlin_Internal, txt ")
632 # LOOP OVER COLUMNS
633 ap.append("! Loop over Columns ")
634 ap.append(f"*DO, ii, 1, {CM_dict[0]}, 1 ")
635 ap.append(" CMSEL, S, COLUMN_%ii% ")
636 ap.append(" ESSL, S ")
637 ap.append(" ESEL, R, ENAME, , 189 ")
638 # FORMAT
639 ap.append(" *IF, ii, LT, 10, THEN ")
640 ap.append(" *VWRITE, ii ")
641 ap.append(" ("NS ColMember_", F3.0) ')
642 ap.append(" *ELSE ")
643 ap.append(" *VWRITE, ii ")
644 ap.append(" ("NS ColMember_", F4.0) ')
645 ap.append(" *ENDIF ")
646 # RESULT
647 ap.append(" *GET, nElem, ELEM, 0, COUNT ")
648 ap.append(" *VWRITE, 'ElemID', 'NF [N]', 'My [Nmm]', 'Mz [Nmm]', 'Vy [N]', 'Vz [N]', 'T [N/mm]', 'Y_LOC' ")
649 ap.append(" (A12, A20, A20, A20, A20, A20, A20, A20) ")
650 ap.append(" ELEM = 0 ")
651 ap.append(" *DO, jj, 1, nElem, 1 ")
652 ap.append(" ELEM = ELNEXT(ELEM) ")
653 ap.append(" *GET, NF, ELEM, ELEM, SMISC, 1 ")
654 ap.append(" *GET, MY, ELEM, ELEM, SMISC, 2 ")
655 ap.append(" *GET, MZ, ELEM, ELEM, SMISC, 3 ")
656 ap.append(" *GET, VY, ELEM, ELEM, SMISC, 6 ")
657 ap.append(" *GET, VZ, ELEM, ELEM, SMISC, 5 ")
658 ap.append(" *GET, TQ, ELEM, ELEM, SMISC, 4 ")
659 ap.append(" NSLE ")
660 ap.append(" *GET, Y_LOC, NODE, 0, MNLOC, Y ")
661 ap.append(" *VWRITE, ELEM, NF, MY, MZ, VY, VZ, TQ, Y_LOC ")
662 ap.append(" (F12.0, 7E20.8) ")
663 ap.append(" *ENDDO ")
664 ap.append(" *ENDDO ")
665
666 # LOOP OVER HORIZONTAL BRACES
667 ap.append("! Loop over Horizontal Braces ")
668 ap.append(f"*DO, ii, 1, {CM_dict[1]}, 1 ")
669 ap.append(" CMSEL, S, HORIZ_%ii%, LINE ")
670 ap.append(" ESSL, S ")
671 ap.append(" ESEL, R, ENAME, , 189 ")
672 # FORMAT
673 ap.append(" *IF, ii, LT, 10, THEN ")
674 ap.append(" *VWRITE, ii ")
675 ap.append(" ("NS HorizMember_", F2.0) ')
676 ap.append(" *ELSE ")
677 ap.append(" *VWRITE, ii ")
678 ap.append(" ("NS HorizMember_", F3.0) ')
679 ap.append(" *ENDIF ")
680 # RESULT
681 ap.append(" *GET, nElem, ELEM, 0, COUNT ")
682 ap.append(" *VWRITE, 'ElemID', 'NF [N]', 'My [Nmm]', 'Mz [Nmm]', 'Vy [N]', 'Vz [N]', 'T [N/mm]', 'Y_LOC' ")

```

```

683 ap.append(" (A12,7A20) ")
684 ap.append(" elem = 0 ")
685 ap.append(" *DO,jj,1,nElem,1 ")
686 ap.append(" ELEM = ELNEXT(ELEM) ")
687 ap.append(" *GET,NF,ELEM,ELEM,SMISC,1 ")
688 ap.append(" *GET,MY,ELEM,ELEM,SMISC,2 ")
689 ap.append(" *GET,MZ,ELEM,ELEM,SMISC,3 ")
690 ap.append(" *GET,VY,ELEM,ELEM,SMISC,6 ")
691 ap.append(" *GET,VZ,ELEM,ELEM,SMISC,5 ")
692 ap.append(" *GET,TQ,ELEM,ELEM,SMISC,4 ")
693 ap.append(" NSLE ")
694 ap.append(" *GET,Y_LOC,NODE,0,MNLOC,Y ")
695 ap.append(" *VWRITE,ELEM,NF,MY,MZ,VY,VZ,TQ,Y_LOC ")
696 ap.append(" (F12.0,7E20.8) ")
697 ap.append(" *ENDDO ")
698 ap.append(" *ENDDO ")
699
700 # LOOP OVER CROSS BRACES
701 ap.append("! Loop over Cross Braces ")
702 ap.append(f"*DO,ii,1,{CM_dict[2]},1 ")
703 ap.append(" CMSEL,S,CROSS_%ii%,LINE ")
704 ap.append(" ESLL,S ")
705 ap.append(" ESEL,R,ENAME,,189 ")
706 # FORMAT
707 ap.append(" *IF,ii,LT,10,THEN ")
708 ap.append(" *VWRITE,ii ")
709 ap.append(" ("NS CrossMember_",F2.0) ")
710 ap.append(" *ELSE ")
711 ap.append(" *VWRITE,ii ")
712 ap.append(" ("NS CrossMember_",F3.0) ")
713 ap.append(" *ENDIF ")
714 # RESULT
715 ap.append(" *GET,nElem,ELEM,0,COUNT ")
716 ap.append(" *VWRITE,'ElemID','NF [N]','My [Nmm]','Mz [Nmm]','Vy [N]','Vz [N]','T [N/mm]','Y_LOC' ")
717 ap.append(" (A12,7A20) ")
718 ap.append(" elem = 0 ")
719 ap.append(" *DO,jj,1,nElem,1 ")
720 ap.append(" ELEM = ELNEXT(ELEM) ")
721 ap.append(" *GET,NF,ELEM,ELEM,SMISC,1 ")
722 ap.append(" *GET,MY,ELEM,ELEM,SMISC,2 ")
723 ap.append(" *GET,MZ,ELEM,ELEM,SMISC,3 ")
724 ap.append(" *GET,VY,ELEM,ELEM,SMISC,6 ")
725 ap.append(" *GET,VZ,ELEM,ELEM,SMISC,5 ")
726 ap.append(" *GET,TQ,ELEM,ELEM,SMISC,4 ")
727 ap.append(" NSLE ")
728 ap.append(" *GET,Y_LOC,NODE,0,MNLOC,Y ")
729 ap.append(" *VWRITE,ELEM,NF,MY,MZ,VY,VZ,TQ,Y_LOC ")
730 ap.append(" (F12.0,7E20.8) ")
731 ap.append(" *ENDDO ")
732 ap.append(" *ENDDO ")
733 ap.append(" *CFCLOSE ")
734
735
736
737 return ap
738
739
740

```